

PHAROS UNIVERSITY IN ALEXANDRIA*Faculty of Computer Science & Artificial Intelligence***Hybrid Forensic Emotion Recognition System***Pattern Recognition*

Student	Alaa abd elhalim elrawy
ID	202303181
Domain	Pattern Recognition
Instructors	Dr. Lamiaa Saaed Eng. Shrouk Farg

1. Abstract	3
2. System Architecture Overview	3
3. Phase I: Pre-Processing (prep.py)	3
3.1 Gamma Correction (gamma = 1.2)	3
3.2 Face Alignment via MediaPipe	3
3.3 CLAHE — Contrast Limited Adaptive Histogram Equalization	4
3.4 Border Fill Strategy	4
4. Phase II: Feature Extraction (fa.py)	4
4.1 Discrete Wavelet Transform (DWT — Haar)	4
4.2 PCA — Dimensionality Reduction	5
5. Phase III: Model Training (model.py)	6
5.1 Class Imbalance and SMOTE	6
5.2 Preventing Data Leakage	7
5.3 Random Forest Tuning	7
6. Phase IV: Orchestration and Visualization (main.py & vis.py)	7
6.1 Main Orchestrator (main.py)	7
6.2 Visualization (vis.py)	7
7. Phase V: Deployment (app.py)	9
7.1 Consistent Pre-Processing	9
7.2 Confidence Levels	9
7.3 Application Screenshots	9
8. Results and Performance	10
8.1 Final Accuracy	10
8.2 Per-Class Results	10
8.3 Observations	11
9. Conclusion	11

1. Abstract

This report describes a system for classifying facial expressions into seven emotion categories: Anger, Contempt, Disgust, Fear, Happy, Sadness, and Surprise. The system is built using a hybrid pipeline that combines classical image processing techniques with machine learning.

The pipeline uses the Discrete Wavelet Transform (DWT) for feature extraction, followed by Principal Component Analysis (PCA) to reduce the number of features, and a Random Forest classifier for the final prediction. This combination results in a system that is both efficient and accurate across all seven emotion classes.

The project is organized into six Python modules: `prep.py`, `fa.py`, `model.py`, `main.py`, `vis.py`, and `app.py`. Each module handles one part of the pipeline, making it easy to test, update, or replace any component independently.

2. System Architecture Overview

The system is organized into five sequential phases, each handled by a dedicated Python module. This structure makes the code clean and easy to maintain — each module does one specific job.

Phase	Module	Responsibility
Phase I	<code>prep.py</code>	Pre-processing: Gamma correction, face alignment, CLAHE enhancement
Phase II	<code>fa.py</code>	Feature Extraction: DWT decomposition and PCA dimensionality reduction
Phase III	<code>model.py</code>	Model Training: SMOTE balancing and Random Forest with GridSearchCV
Phase IV	<code>main.py</code> / <code>vis.py</code>	Orchestration and Visualization: data splits, model export, confusion matrix
Phase V	<code>app.py</code>	Deployment: Streamlit web application for real-time inference

3. Phase I: Pre-Processing (`prep.py`)

Before any feature extraction, each input image goes through a four-step pre-processing pipeline. This ensures that factors like lighting conditions or camera angle do not negatively affect the results.

3.1 Gamma Correction (`gamma = 1.2`)

Images taken in low-light environments can have dark or uneven brightness. Gamma correction adjusts the brightness using the following formula:

$$V_{out} = V_{in} ^ \text{gamma} \quad (\text{gamma} = 1.2)$$

A gamma value of 1.2 was chosen to slightly brighten the image without washing out the highlights, making facial features more visible.

3.2 Face Alignment via MediaPipe

The system uses MediaPipe to detect two eye landmark points (33 and 263). It then calculates the angle between the eyes and rotates the face so it is perfectly upright. This is important because the DWT feature extraction is position-sensitive — the same facial region must always appear at the same location in every image.

3.3 CLAHE — Contrast Limited Adaptive Histogram Equalization

CLAHE divides the image into small 8x8 tiles and adjusts the contrast of each tile separately. This helps make subtle facial details (like slight changes around the mouth or eyes) more visible, without adding noise to the image.

3.4 Border Fill Strategy

When the face is rotated, the edges of the image can become empty. The system fills these gaps by repeating the nearest border pixel (cv2.BORDER_REPLICATE). This avoids creating fake edges that could interfere with the wavelet analysis.

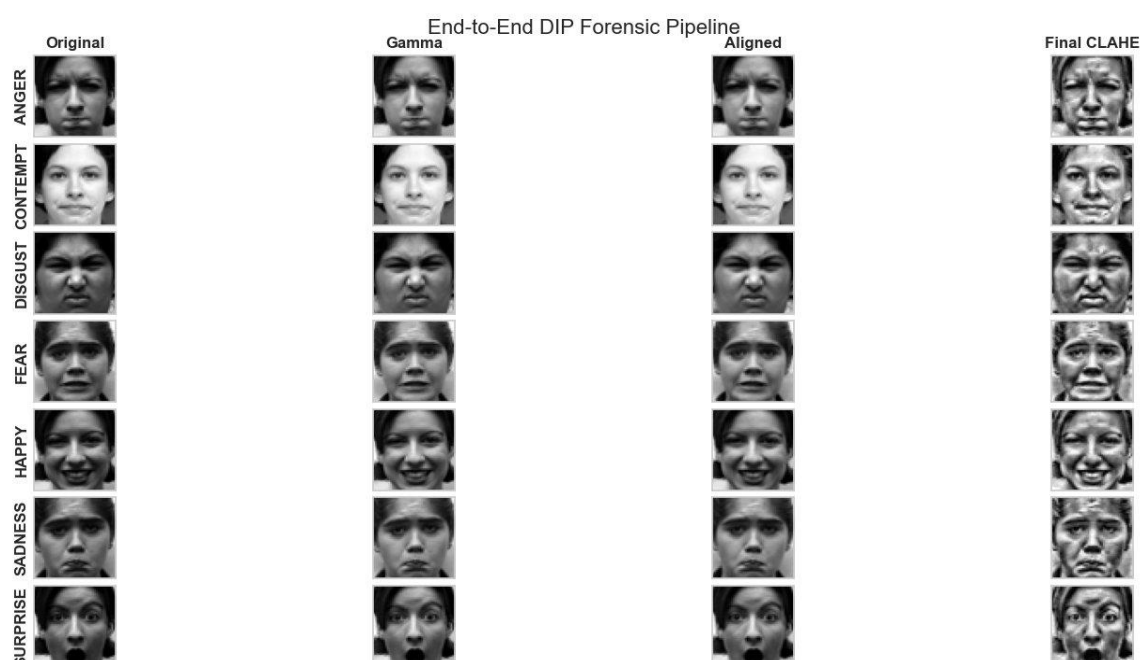


Figure 1 — Pre-Processing Pipeline: Original, Gamma Corrected, Aligned, and Final CLAHE (shown for all seven emotion classes)

4. Phase II: Feature Extraction (fa.py)

A raw 48x48 image contains 2,304 pixel values. Many of these are redundant or irrelevant. This phase converts the image into a smaller, more meaningful set of features that the classifier can work with more effectively.

4.1 Discrete Wavelet Transform (DWT — Haar)

The Haar Wavelet Transform splits each image into four sub-bands:

- LL (Approximation): The low-frequency summary of the image. This captures the overall shape and structure of the face, which is what changes between different emotions.
- LH (Horizontal Detail): Horizontal edges like eyebrow lines and lip boundaries.
- HL (Vertical Detail): Vertical edges like the sides of the nose and cheeks.
- HH (Diagonal Detail): High-frequency noise from skin texture and camera sensors.

Only the LL sub-band is kept. The other three bands mostly contain noise and texture details that are not useful for emotion classification. Keeping only the LL band reduces the image from 48x48 to 24x24, cutting the number of features by 75%.

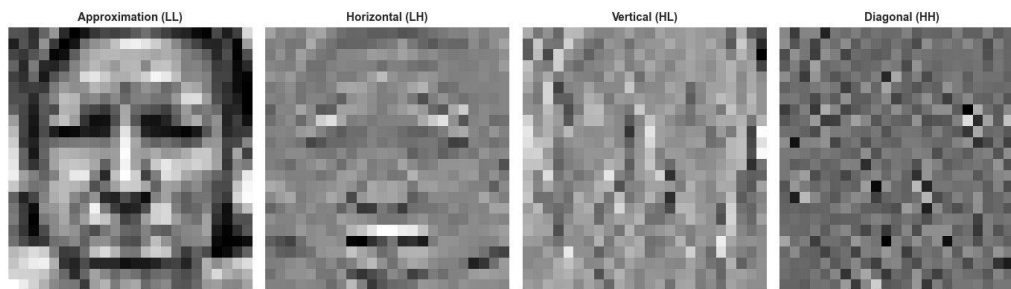


Figure 2 — DWT Sub-band Decomposition: Approximation (LL), Horizontal (LH), Vertical (HL), and Diagonal (HH). Only the LL sub-band is used for classification.

4.2 PCA — Dimensionality Reduction

The 576 values from the 24x24 LL image are further compressed using Principal Component Analysis (PCA):

- The system keeps only the components needed to explain 95% of the data variation. This turns out to be approximately 142 components out of 576.
- Whitening is also applied, which makes all features have equal scale. This helps the Random Forest learn more evenly from all features.

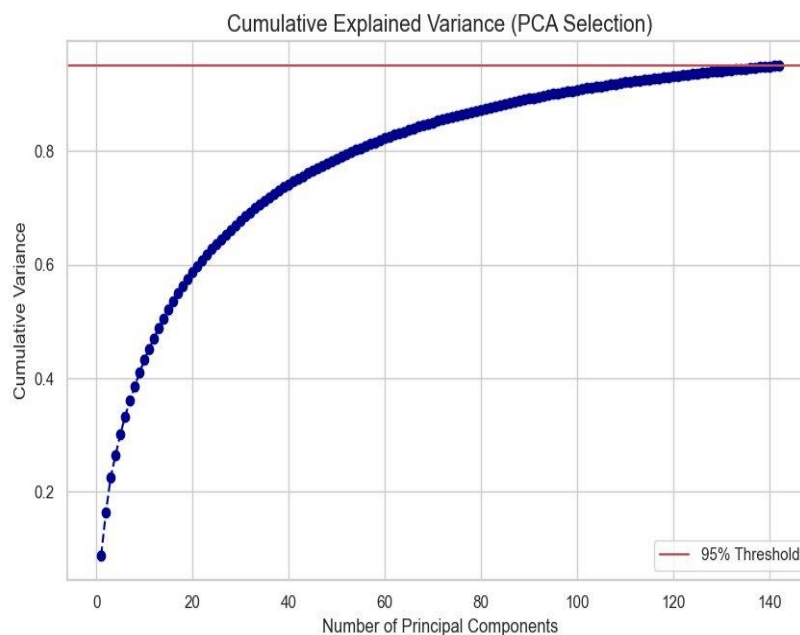


Figure 3 — Cumulative Explained Variance. The red line marks the 95% threshold, which is reached at approximately 142 principal components.

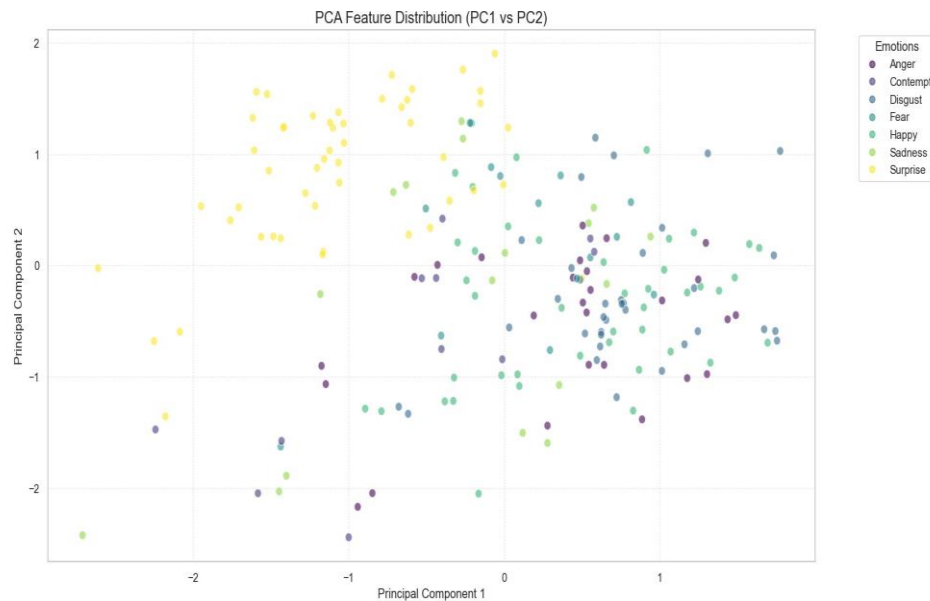


Figure 4 — PCA Feature Space (PC1 vs PC2). The Surprise class (yellow) is clearly separated from the other emotions in the upper-left area, which helps explain its perfect classification accuracy.

5. Phase III: Model Training (model.py)

5.1 Class Imbalance and SMOTE

In the original dataset, some emotions have far more samples than others. For example, Surprise has many more images than Contempt or Fear. If the model is trained on this imbalanced data, it will tend to perform better on common emotions and worse on rare ones.

To fix this, SMOTE (Synthetic Minority Over-sampling Technique) is used. SMOTE creates new synthetic samples for the minority classes by interpolating between existing samples. After SMOTE, all seven classes have exactly 157 samples each.

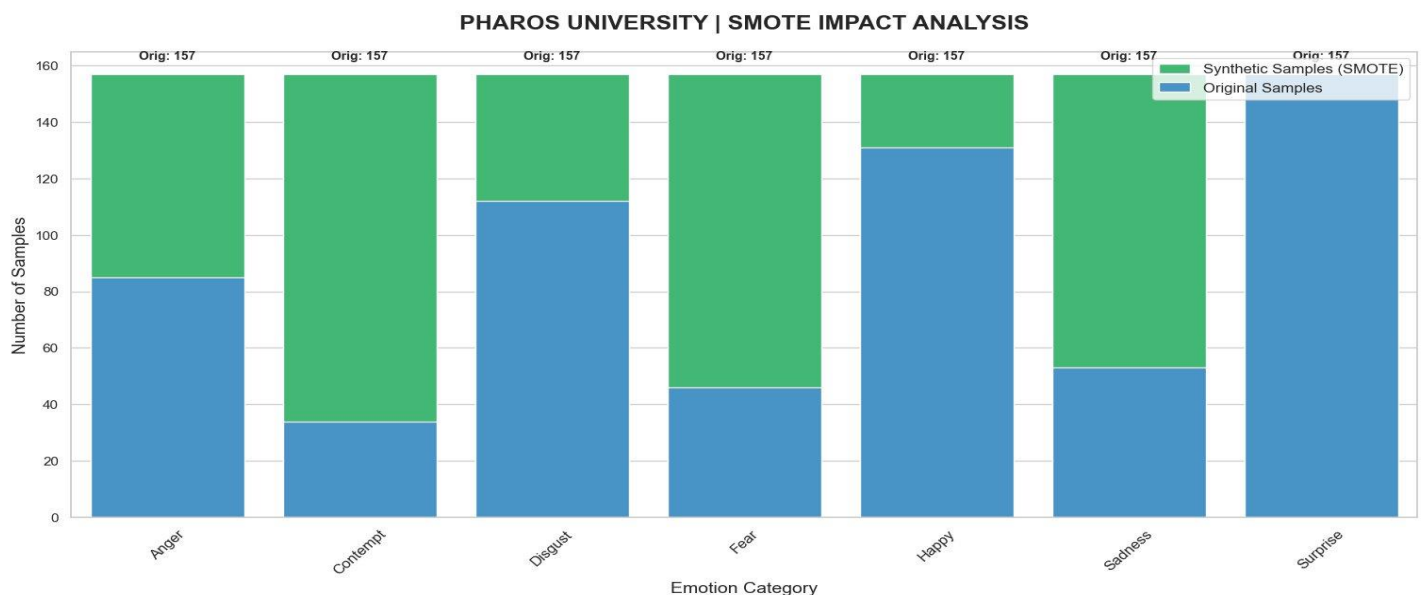


Figure 5 — SMOTE Impact Analysis: class distribution before and after balancing. All classes reach 157 samples after SMOTE is applied.

5.2 Preventing Data Leakage

SMOTE is applied inside the training pipeline, not outside it. This means synthetic samples are only created from training data during each cross-validation fold. The test data is never used or seen during this process, which keeps the evaluation results honest and reliable.

5.3 Random Forest Tuning

The Random Forest classifier is tuned using GridSearchCV with 5-fold cross-validation. The best configuration found was:

- 500 trees in the ensemble.
- Max depth of 20 per tree.
- Min samples per leaf: 2, min samples per split: 5.
- SMOTE k neighbors: 3.

The best cross-validation F1-Macro score achieved was 0.8736.

6. Phase IV: Orchestration and Visualization (main.py & vis.py)

6.1 Main Orchestrator (main.py)

The main module handles the full training workflow. It splits the data 80/20 (training/testing), runs the pipeline, and saves the trained PCA and Random Forest models as .pkl files. These files are later loaded by the application for real-time prediction.

6.2 Visualization (vis.py)

Two visual outputs are generated after training:

- Confusion Matrix: Shows how well the model classifies each emotion. The matrix in Figure 6 shows that most errors happen between similar-looking emotions (Disgust confused with Happy: 3 cases; Fear confused with Surprise: 2 cases; Sadness confused with Surprise: 1 case).
- SMOTE Chart: Shows the number of original vs. synthetic samples per class, confirming that all classes are balanced before training.

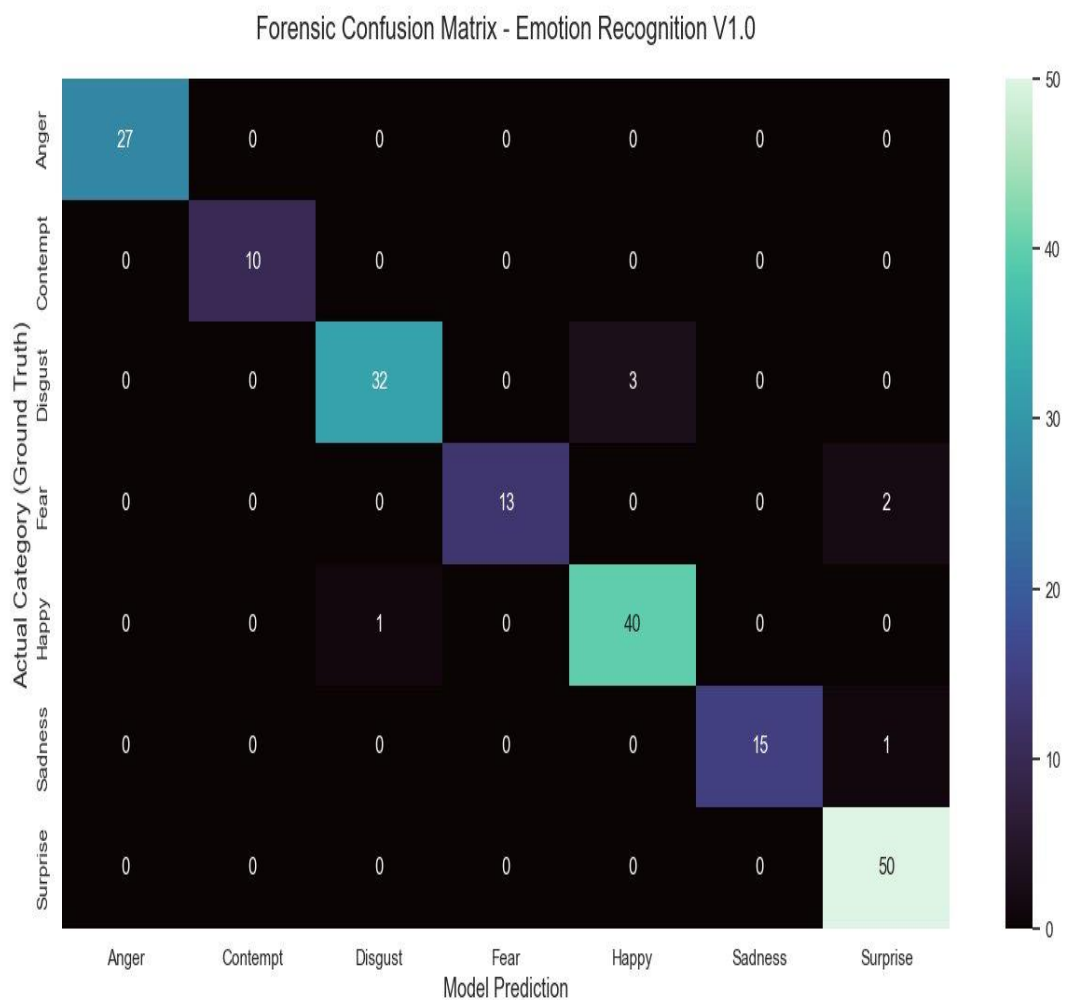


Figure 6 — Confusion Matrix (Emotion Recognition). Strong diagonal values indicate high accuracy. Minor confusions: Disgust/Happy (3), Fear/Surprise (2), Sadness/Surprise (1).

7. Phase V: Deployment (app.py)

The trained model is deployed as a web application using Streamlit, allowing users to upload a face image and receive an emotion prediction in real time.

7.1 Consistent Pre-Processing

The application applies the exact same pre-processing steps used during training: Gamma correction, face alignment, CLAHE, DWT, and PCA. This ensures the model receives input data in the same format it was trained on, which is critical for maintaining accuracy in production.

7.2 Confidence Levels

Each prediction comes with a confidence score based on the model's class probabilities:

Confidence Level	Probability	Meaning
High	70% or above	Reliable prediction
Moderate	45% to 69%	Prediction is acceptable but uncertain
Low	Below 45%	Unreliable — re-capture the image

7.3 Application Screenshots

The screenshots below show the main interface of the deployed application.

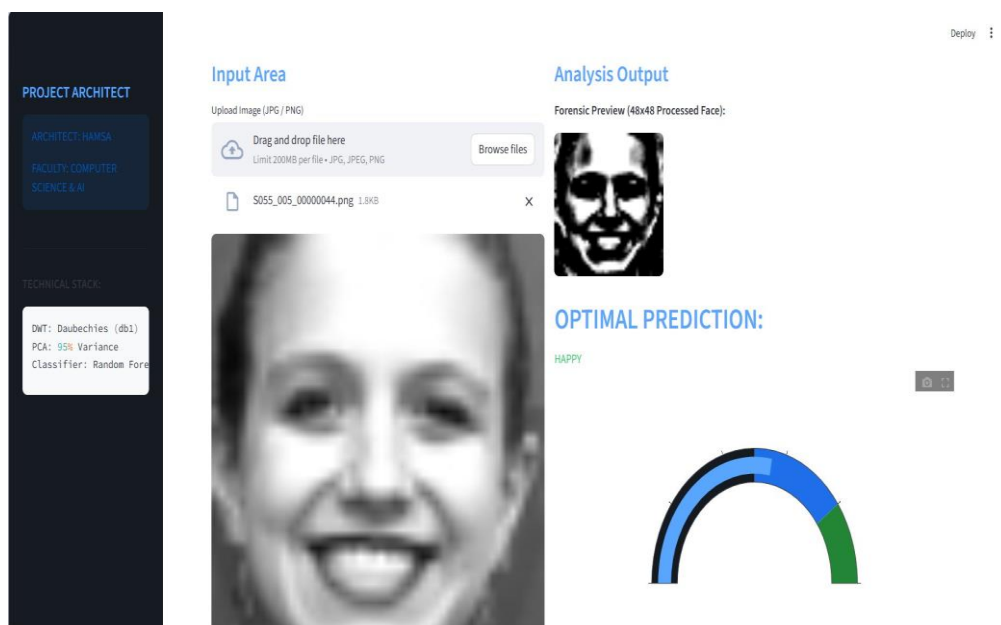


Figure 7 — Application Interface: image upload area, 48x48 processed preview, and emotion prediction with confidence gauge.



Figure 8 — Probability Distribution panel: probability bars for all seven classes, confidence level (Moderate, 55.77%), and prediction summary.

8. Results and Performance

8.1 Final Accuracy

The model was tested on 194 samples (20% of the full dataset). The final results were:

- Overall Accuracy: 96.39%
- F1-Macro Score: 0.9658
- Macro Average Precision: 0.98 | Recall: 0.96 | F1: 0.97

```
(venv) PS D:\Pattren> python main.py
warnings.warn(

[OK] Best Parameters: {'rf_max_depth': 20, 'rf_min_samples_leaf': 2, 'rf_min_samples_split': 5, 'rf_n_estimators': 500, 'smote_k_neighbors': 3}
[OK] Best CV Score (F1-Macro): 0.8736

!!!!!!!!!!!!!!!!!!!!!! FINAL REPORT !!!!!!!!!!!!!!!!!!!!!!!

[FINAL ACCURACY] 96.39%
[FINAL F1-MACRO] 0.9658

[CLASSIFICATION REPORT]
      precision    recall  f1-score   support

   Anger         1.00      1.00      1.00        27
  Contempt       1.00      1.00      1.00        10
   Disgust       0.97      0.91      0.94        35
     Fear       1.00      0.87      0.93        15
    Happy       0.93      0.98      0.95        41
   Sadness       1.00      0.94      0.97        16
   Surprise      0.94      1.00      0.97        50

 accuracy          0.96          0.96        194
  macro avg       0.98          0.96          0.97        194
 weighted avg     0.97          0.96          0.96        194
```

Figure 9 — Terminal output showing final model accuracy (96.39%), F1-Macro score (0.9658), and the full per-class classification report.

8.2 Per-Class Results

The table below summarizes the performance for each emotion class based on the classification report:

Emotion	Precision	Recall	F1-Score	Support
Anger	1.00	1.00	1.00	27
Contempt	1.00	1.00	1.00	10
Disgust	0.97	0.91	0.94	35
Fear	1.00	0.87	0.93	15
Happy	0.93	0.98	0.95	41
Sadness	1.00	0.94	0.97	16
Surprise	0.94	1.00	0.97	50

8.3 Observations

Anger and Contempt achieved perfect scores across all metrics. Surprise had perfect recall (50/50), which matches the clear cluster separation visible in the PCA distribution plot. The only notable errors were between Disgust and Happy (3 cases), Fear and Surprise (2 cases), and Sadness and Surprise (1 case) — all pairs that share some visual similarity in facial structure.

9. Conclusion

This project demonstrates that combining classical image processing techniques with machine learning can produce a highly accurate facial emotion recognition system. The pipeline achieved 96.39% overall accuracy and an F1-Macro score of 0.9658 across seven emotion classes.

The key design choices that contributed to this performance were:

- Face alignment ensures consistent feature positions across all images.
- DWT LL sub-band selection removes noise while keeping the important facial structure.
- SMOTE balancing inside the training pipeline prevents biased results on minority classes.
- Saving the trained models as .pkl files ensures the app uses the exact same processing as training.

Future improvements could include using a deeper wavelet decomposition, adding real-time video support, or testing the system on more diverse face datasets.